

Evaluación Integrada de Desempeño N°1

Análisis y Modelamiento de Secciones y Funciones por **Cónicas** Tramos a partir del RUT

Curso: MAT1186 — Introducción al Cálculo

Profesores: Higuera, N. Estrada, C.

fecha: 29/05/2026

Integrantes del Grupo

Líder: Benjamin Alexander Cantero Garrido

Integrante 2: Eduardo Antonio Domínguez Finschi

Integrante 3: Ricardo Andrés Garcés Álvarez

Contents

1	Introducción	2
1.1	Objetivo general	2
1.2	Restricciones de implementación	2
1.3	Arquitectura del proyecto	2
1.4	Capa de servicios y manejo de errores	3
2	Módulo de Validación del RUT (core/rut.py)	4
2.1	Fundamento matemático: algoritmo módulo 11	4
2.2	Implementación de la validación	4
2.2.1	Ejemplo de validación	5
3	Módulo de Secciones Cónicas (core/conica.py)	6
3.1	Fundamento matemático	6
3.1.1	Ecuación general de segundo grado	6
3.1.2	Ajustes determinísticos sobre los coeficientes	6
3.1.3	Clasificación de cónicas	7
3.2	Modelos matemáticos auxiliares (core/modelos.py)	8
3.2.1	Completar el cuadrado	8
3.2.2	Raíz cuadrada: método de Newton-Raphson	8
3.2.3	Funciones trigonométricas e hiperbólicas: series de Taylor	9
3.3	Forma canónica y procedimiento inverso	10
3.3.1	Circunferencia	10
3.3.2	Elipse e hipérbola	10
3.3.3	Parábola	10
4	Módulo de Límites y Funciones por Tramos (core/limites.py)	11
4.1	Fundamento matemático: Límites laterales	11
4.2	Construcción de la función por tramos	11
4.3	Tipos de discontinuidad	11
4.3.1	Discontinuidad removible	11
4.3.2	Discontinuidad de salto	11
4.3.3	Discontinuidad infinita	12
4.4	Evidencia computacional: tabla de valores	12
4.4.1	Ejemplo de tabla (RUT 12.345.678-5, $a = 3$, caso infinita)	13
5	Validadores, Constantes y Trazabilidad	13
5.1	Validadores reutilizables (core/validators.py)	13
5.2	Constantes (core/constants.py)	14
5.3	Sistema de logging (core/log_service.py)	14
6	Módulo de Interfaz Gráfica (ui/)	14
6.1	Tecnología utilizada: Tkinter	14
6.2	Estructura de la interfaz	14
6.3	Graficación sin librerías matemáticas	15
6.4	Campos de defensa oral	15

7 Pruebas (test_services.py)	15
8 RUTs de Prueba Verificados	16
9 Cumplimiento de Requerimientos	16
10 Conclusiones	16
Referencias	17

1. Introducción

El presente informe documenta el proceso de investigación, diseño e implementación del proyecto correspondiente a la Evaluación Integrada de Desempeño N°1 del curso MAT1186 — Introducción al Cálculo. El sistema desarrollado integra conceptos de geometría analítica, límites y programación en Python, sobre una arquitectura modular que separa la lógica matemática (capa `core/`) de la interfaz gráfica (capa `ui/`).

1.1. Objetivo general

Desarrollar una aplicación en Python que permita, a partir de un RUT chileno válido:

1. Validar el RUT mediante el algoritmo oficial del módulo 11.
2. Construir una ecuación general de segundo grado y clasificar la cónica obtenida.
3. Transformar la ecuación a su forma canónica, mostrando el procedimiento completo en ambos sentidos (general $\rightarrow$ canónica y canónica $\rightarrow$ general).
4. Analizar funciones por tramos generadas desde el RUT, calculando límites laterales y clasificando discontinuidades, con evidencia numérica mediante tabla de valores.

1.2. Restricciones de implementación

Según los requerimientos del proyecto, se prohíbe el uso de librerías matemáticas como `numpy`, `math`, `sympy`, `scipy` o `pandas`. **Todos los cálculos fueron implementados manualmente.** La única librería externa utilizada es `tkinter`, incluida en la instalación estándar de Python para la interfaz gráfica. El proyecto sí emplea módulos de la biblioteca estándar (`dataclasses`, `typing`, `logging`, `datetime`) que no realizan cálculo matemático, sino que organizan datos y trazabilidad.

1.3. Arquitectura del proyecto

El proyecto adopta una arquitectura por capas. La capa `core/` concentra toda la lógica matemática y de negocio, mientras que `ui/` se ocupa exclusivamente de la presentación. Entre ambas se intercala una **capa de servicios** (`core/services.py`) que orquesta el flujo completo (validar $\rightarrow$ calcular $\rightarrow$ clasificar $\rightarrow$ transformar) y entrega los resultados como objetos de datos estructurados (`dataclasses`). Esta separación facilita las pruebas y evita duplicar lógica en los paneles de la interfaz.

Estructura de archivos real

```

EID_1/
|-- main.py                # Punto de entrada (lanza la app)
|-- README.md
|-- etica.md               # Código de ética del grupo
|-- test_services.py       # Pruebas de la capa de servicios
|-- core/
|   |-- rut.py             # Validación RUT (módulo 11)
|   |-- conica.py          # Cónicas y forma canónica
|   |-- limites.py         # Funciones por tramos y límites
|   |-- modelos.py         # Modelos matemáticos (raíz, Taylor)
|   |-- graficas.py        # Generación de puntos para graficar
|   |-- services.py        # Capa de servicios (orquestración)
|   |-- validators.py       # Validadores reutilizables
|   |-- exceptions.py       # Excepciones personalizadas
|   |-- constants.py        # Constantes matemáticas
|   |-- config.py          # Configuración visual de la UI
|   '-- log_service.py      # Sistema de logging / trazabilidad
'-- ui/
    |-- app.py              # Aplicación principal (ventana + tabs)
    |-- panel_conica.py     # Panel de cónicas
    |-- panel_limites.py    # Panel de límites
    |-- panel_logs.py       # Panel de registro de eventos
    '-- componentes.py      # Componentes UI reutilizables

```

1.4. Capa de servicios y manejo de errores

Cada panel de la interfaz invoca una única función de servicio ([analizar_conica](#) o [analizar_limites](#)), que devuelve una `dataclass` con todo el análisis. Los errores se modelan con excepciones propias definidas en `core/exceptions.py`: `RUTInvalidoError`, `ConicaInvalidaError` y `LimiteInvalidoError` (todas heredan de `EIDError`). Así, la UI captura una excepción y muestra un mensaje claro al usuario sin mezclar validación con presentación.

```

1  @dataclass
2  class ConicaAnalysis:
3      es_valido: bool = False
4      digitos: list = field(default_factory=list)
5      dv: str = ""
6      A: float = 0.0
7      B: float = 0.0
8      # ... C, D, E, tipo_conica, ecuacion_general,
9          ecuacion_canonica ...
10     elementos_geometricos: dict = field(default_factory=dict)
11  def analizar_conica(rut_str):
12     """Orquesta validacion -> coeficientes -> clasificacion ->
13         canonica."""
14     rut_str = validar_rut_str(rut_str)  #
15         validators.py
16     es_valido, pasos, digitos, dv = validar_rut(rut_str)  #

```

```

rut.py
15 if not es_valido:
16     raise RUTInvalidoError(...) #
    exceptions.py
17 A, B, C, D, E, pasos, ajustes = calcular_coeficientes(digitos,
    dv)
18 tipo = clasificar_conica(A, B)
19 canonica, pasos_can, elementos = forma_canonica(A, B, C, D, E,
    tipo)
20 # ... arma y retorna el objeto ConicaAnalysis ...

```

Listing 1: Servicio de análisis de cónica — `core/services.py` (extracto)

2. Módulo de Validación del RUT (`core/rut.py`)

2.1. Fundamento matemático: algoritmo módulo 11

El dígito verificador de un RUT chileno se calcula mediante el algoritmo oficial del módulo 11, definido por el Servicio de Registro Civil e Identificación de Chile [1]. El procedimiento es el siguiente:

Algoritmo módulo 11 para el RUT chileno:

Sea el RUT $d_1d_2d_3d_4d_5d_6d_7d_8 - DV$.

1. Multiplicar cada dígito del cuerpo de derecha a izquierda por la serie cíclica $[2, 3, 4, 5, 6, 7, 2, 3, 4, \dots]$.
2. Sumar todos los productos obtenidos: $S = \sum_{i=1}^8 d_i \cdot f_i$
3. Calcular $r = S \bmod 11$
4. Obtener el dígito verificador: $DV = 11 - r$
 - Si $11 - r = 11 \Rightarrow DV = 0$
 - Si $11 - r = 10 \Rightarrow DV = K$
 - En otro caso $\Rightarrow DV = 11 - r$

Fuentes de investigación

- [Registro Civil de Chile — Algoritmo oficial del dígito verificador](#)
- [Wikipedia — Número de identificación tributaria \(Chile\)](#)

2.2. Implementación de la validación

La función `validar_rut` recibe el RUT como texto, lo limpia mediante `limpiar_rut` (eliminando puntos y guión), valida el formato básico (longitud y que el cuerpo sea numérico), aplica el algoritmo del módulo 11 y retorna una tupla con el resultado, el procedimiento paso a paso, la lista de dígitos y el dígito verificador calculado.

```

1 def validar_rut(rut_str):
2     """
3     Valida un RUT chileno usando el algoritmo oficial del modulo
4     11.
5     Retorna: (es_valido, pasos, digitos, dv_calculado)
6     Fuente: Registro Civil de Chile - algoritmo modulo 11
7     """

```

```

7      limpio = limpiar_rut(rut_str)                # quita puntos,
           guion, mayuscula
8
9      if len(limpio) < 2:
10         return False, ["RUT demasiado corto."], [], ""
11
12     cuerpo = limpio[:-1]
13     dv_ingresado = limpio[-1]
14
15     if not cuerpo.isdigit():
16         return False, ["El cuerpo debe contener solo digitos."],
            [], ""
17     if len(cuerpo) < 7 or len(cuerpo) > 8:
18         return False, ["El cuerpo debe tener entre 7 y 8
            digitos."], [], ""
19
20     cuerpo_8 = cuerpo.zfill(8)                    # rellena a 8 digitos
21     serie = [2, 3, 4, 5, 6, 7]                    # serie ciclica
           oficial
22     suma = 0
23     pasos = []
24
25     for i, digito in enumerate(reversed(cuerpo_8)):
26         factor = serie[i % 6]
27         producto = int(digito) * factor
28         suma += producto
29         pasos.append(f" {digito} x {factor} = {producto} (suma:
            {suma})")
30
31     resto = suma % 11
32     resultado = 11 - resto
33
34     if resultado == 11:
35         dv_calculado = "0"
36     elif resultado == 10:
37         dv_calculado = "K"
38     else:
39         dv_calculado = str(resultado)
40
41     es_valido = (dv_calculado == dv_ingresado)
42     digitos = [int(c) for c in cuerpo_8]
43     return es_valido, pasos, digitos, dv_calculado

```

Listing 2: Función de validación del RUT — `core/rut.py`

2.2.1. Ejemplo de validación

Para el RUT **12.345.678-5**:

Dígito	Factor	Producto	Suma acum.
8	2	16	16
7	3	21	37
6	4	24	61
5	5	25	86
4	6	24	110
3	7	21	131
2	2	4	135
1	3	3	138

$$S = 138, \quad r = 138 \bmod 11 = 6, \quad DV = 11 - 6 = 5 \checkmark$$

El módulo también ofrece `formatear_rut`, que reinserta los puntos y el guión para mostrar el RUT al usuario.

3. Módulo de Secciones Cónicas (`core/conica.py`)

3.1. Fundamento matemático

3.1.1. Ecuación general de segundo grado

Una sección cónica en el plano cartesiano (sin término xy) puede representarse mediante la ecuación general [7]:

$$Ax^2 + By^2 + Cx + Dy + E = 0$$

donde los coeficientes *base* se construyen a partir de los dígitos del RUT:

$$A = \frac{d_1 + d_2}{v}, \quad B = \frac{d_3 + d_4}{v}, \quad C = -(d_5 + d_6), \quad D = -(d_7 + d_8), \quad E = d_1 + d_3 + d_5 + d_7$$

y la variable auxiliar v se define como:

$$v = \begin{cases} 10 & \text{si } DV = K \\ 11 & \text{si } DV = 0 \\ DV & \text{si } DV \in \{1, \dots, 9\} \end{cases}$$

Como v nunca puede ser cero, antes de calcular se verifica esta condición y, de ser necesario, se lanza `ConicaInvalidaError` para evitar una indeterminación.

3.1.2. Ajustes determinísticos sobre los coeficientes

Para garantizar que el conjunto de RUTs cubra las cuatro cónicas exigidas, tras calcular los coeficientes base la función `calcular_coeficientes` aplica una serie de ajustes determinísticos en función de los dígitos. Cada ajuste se registra en la lista `ajustes` para poder mostrarlo y explicarlo durante la defensa:

Condición	Ajuste aplicado	Tendencia
d_8 impar	$B \leftarrow -B$	favorece hipérbola
$d_1 = d_2$	$B \leftarrow A$	favorece circunferencia
$(d_5 + d_6) \bmod 3 = 0$, d_7 par	$B \leftarrow 0$	parábola eje vertical
$(d_5 + d_6) \bmod 3 = 0$, d_7 impar	$A \leftarrow 0$	parábola eje horizontal

```

1 def calcular_coeficientes(digitos, dv_str):
2     """Construye A, B, C, D, E y aplica ajustes determinísticos."""
3     d = digitos
4     v = 10 if dv_str == "K" else (11 if dv_str == "0" else
5         int(dv_str))
6     if v == 0:
7         raise ConicaInvalidaError(mensaje="Indeterminacion: v =
8             0", ...)
9
10    A = (d[0] + d[1]) / v
11    B = (d[2] + d[3]) / v
12    C = -(d[4] + d[5])
13    D = -(d[6] + d[7])
14    E = d[0] + d[2] + d[4] + d[6]
15
16    ajustes = []
17    if d[7] % 2 != 0:                                # d8 impar -> hiperbola
18        B = -B
19        ajustes.append(f"d8={d[7]} impar -> B = -B")
20    if d[0] == d[1]:                                  # d1 == d2 -> circunferencia
21        B = A
22        ajustes.append(f"d1==d2 -> B = A")
23    if (d[4] + d[5]) % 3 == 0:                         # parabola
24        if d[6] % 2 == 0:
25            B = 0
26        else:
27            A = 0
28    return A, B, C, D, E, pasos, ajustes

```

Listing 3: Construcción y ajuste de coeficientes — `core/conica.py`

3.1.3. Clasificación de cónicas

La clasificación depende de los signos y valores finales de A y B [2]:

Tipo	Condición	Forma canónica
Circunferencia	$A = B \neq 0$	$(x - h)^2 + (y - k)^2 = r^2$
Elipse	$A \neq B$, mismo signo	$\frac{(x - h)^2}{a^2} + \frac{(y - k)^2}{b^2} = 1$
Hipérbola	A, B con signos opuestos	$\frac{(x - h)^2}{a^2} - \frac{(y - k)^2}{b^2} = 1$
Parábola	$A = 0$ o $B = 0$	$y = p(x - h)^2 + k$

```

1 def clasificar_conica(A, B):
2     """Clasifica la conica segun los coeficientes A y B."""
3     if A == 0 and B == 0:
4         return "Degenerada"
5     if A == 0 or B == 0:
6         return "Parábola"

```

```

7     if abs(A - B) < 1e-9:                                # A == B con
        tolerancia
8         return "Circunferencia"
9     if (A > 0 and B > 0) or (A < 0 and B < 0):
10        return "Elipse"
11    return "Hipérbola"                                    # signos opuestos

```

Listing 4: Clasificación de la cónica — `core/conica.py`

Fuentes de investigación

- Khan Academy — Secciones cónicas (español)
- OpenStax — Álgebra y Trigonometría (Capítulos 10-11, gratuito)
- Wikipedia — Sección cónica

3.2. Modelos matemáticos auxiliares (`core/modelos.py`)

Las operaciones que normalmente provee `math` se implementan manualmente en `core/modelos.py` y se importan desde `core/conica.py` y `core/graficas.py`.

3.2.1. Completar el cuadrado

La transformación de la forma general a la canónica requiere completar el cuadrado. Para una expresión $\text{coef}(x^2 + \frac{\text{lin}}{\text{coef}}x)$, el procedimiento es [3]:

$$\text{coef}\left(x^2 + \frac{\text{lin}}{\text{coef}}x\right) = \text{coef}\left(x + \frac{\text{lin}}{2\text{coef}}\right)^2 - \frac{\text{lin}^2}{4\text{coef}}$$

El vértice o centro en x queda: $h = -\frac{\text{lin}}{2\text{coef}}$

```

1  def completar_cuadrado(coef, lin):
2      """
3      Completa el cuadrado de: coef*(x^2 + (lin/coef)*x)
4      Retorna (h, adicional) donde h es el centro/vertice y
5      'adicional' es el termino que se traslada al otro lado.
6      Referencia: Khan Academy - Completing the square
7      """
8      h = -lin / (2 * coef)
9      adicional = -(lin * lin) / (4 * coef)
10     return h, adicional

```

Listing 5: Completar el cuadrado — `core/modelos.py`

3.2.2. Raíz cuadrada: método de Newton-Raphson

Para calcular raíces cuadradas sin `math.sqrt()` se implementó el **método de Newton-Raphson** (método de Herón) [8]:

Dado $n \geq 0$, la sucesión $x_{k+1} = \frac{1}{2}\left(x_k + \frac{n}{x_k}\right)$, con $x_0 = \frac{n}{2}$, converge a $\sqrt{n}$.

```

1 def raiz_cuadrada_manual(n):
2     """
3     Calcula raiz cuadrada con el metodo de Newton-Raphson (Heron).
4     Converge cuadraticamente a sqrt(n). Retorna None si n < 0.
5     Referencia: Wikipedia - Metodo de Newton
6     """
7     if n < 0:
8         return None
9     if n == 0:
10        return 0.0
11    x = n / 2.0
12    for _ in range(100):
13        x_nuevo = (x + n / x) / 2.0
14        if abs(x_nuevo - x) < 1e-10:
15            return x_nuevo
16        x = x_nuevo
17    return x

```

Listing 6: Raíz cuadrada por Newton-Raphson — [core/modelos.py](#)

3.2.3. Funciones trigonométricas e hiperbólicas: series de Taylor

Para graficar circunferencias y elipses en forma paramétrica, y las ramas de la hipérbola, se implementaron las **series de Maclaurin** [9]. El módulo provee [sin_taylor](#), [cos_taylor](#), [exp_taylor](#), [cosh_taylor](#) y [sinh_taylor](#).

$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!}, \quad \sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!}$$

$$\cosh(x) = \sum_{n=0}^{\infty} \frac{x^{2n}}{(2n)!}, \quad \sinh(x) = \sum_{n=0}^{\infty} \frac{x^{2n+1}}{(2n+1)!}$$

```

1 def sin_taylor(x):
2     """sin(x) = x - x^3/3! + x^5/5! - ... (serie de Maclaurin)."""
3     pi2 = 6.283185307179586 # 2*pi manual
4     x = x % pi2 # reduce al rango [-pi, pi]
5     if x > 3.141592653589793:
6         x -= pi2
7     result = x
8     term = x
9     for n in range(1, 9): # 9 terminos -> alta precision
10        term *= -x * x / ((2 * n) * (2 * n + 1))
11        result += term
12    return result
13
14 def cos_taylor(x):
15     """cos(x) = 1 - x^2/2! + x^4/4! - ..."""
16     pi2 = 6.283185307179586
17     x = x % pi2
18     if x > 3.141592653589793:
19         x -= pi2
20     result = 1.0
21     term = 1.0
22     for n in range(1, 9):

```

```

23     term *= -x * x / ((2 * n - 1) * (2 * n))
24     result += term
25     return result

```

Listing 7: Seno y coseno por series de Taylor — [core/modelos.py](#)

Fuentes de investigación

- [Wikipedia — Método de Newton-Raphson](#)
- [Khan Academy — Series de Taylor y Maclaurin](#)
- [Wikipedia — Serie de Taylor](#)
- [MIT OpenCourseWare — Single Variable Calculus](#)

3.3. Forma canónica y procedimiento inverso

La función `forma_canonica` despacha, según el tipo de cónica, a una rutina especializada (`_canonica_circunferencia`, `_canonica_elipse`, `_canonica_hiperbola`, `_canonica_parabola`). Cada rutina genera los pasos del procedimiento directo y del inverso, además de un diccionario con los elementos geométricos (centro, radio, semiejes, focos, vértices, directriz).

3.3.1. Circunferencia

Dada $Ax^2 + Ay^2 + Cx + Dy + E = 0$ con $A = B$:

1. Dividir por A para normalizar.
2. Completar el cuadrado en x e y .
3. Identificar h , k y $r^2 = -e_2 - \text{add}_x - \text{add}_y$.
4. Forma canónica: $(x - h)^2 + (y - k)^2 = r^2$, con $r = \text{raiz_cuadrada_manual}(r^2)$.

Procedimiento inverso (canónica $\rightarrow$ general): se expanden los binomios, $x^2 - 2hx + h^2 + y^2 - 2ky + k^2 - r^2 = 0$, y si $A \neq 1$ se multiplica toda la ecuación por A para recuperar los coeficientes originales.

3.3.2. Elipse e hipérbola

Para $Ax^2 + By^2 + Cx + Dy + E = 0$ se completa el cuadrado en ambas variables y se define la constante $K = -E - \text{add}_x - \text{add}_y$. Dividiendo se obtiene $a^2 = K/A$ y $b^2 = K/B$. En la elipse (A, B mismo signo) la distancia focal es $c = \sqrt{|a^2 - b^2|}$; en la hipérbola (A, B signos opuestos), según cuál denominador sea positivo se decide si es horizontal o vertical y se usa $c = \sqrt{a^2 + b^2}$.

3.3.3. Parábola

Cuando $B = 0$ (eje vertical) se completa el cuadrado en x y se despeja $y = p(x - h)^2 + q$ con $p = -A/D$. Cuando $A = 0$ (eje horizontal) se completa en y y se despeja $x = p(y - k)^2 + q$ con $p = -B/C$. En ambos casos se calculan foco ($\pm \frac{1}{4p}$ respecto del vértice) y directriz.

4. Módulo de Límites y Funciones por Tramos (`core/limites.py`)

4.1. Fundamento matemático: Límites laterales

El límite lateral describe el comportamiento de $f(x)$ al acercarse al punto a desde la izquierda o la derecha [4]:

$$\lim_{x \rightarrow a^-} f(x) = L_1, \quad \lim_{x \rightarrow a^+} f(x) = L_2$$

El **límite existe** si y sólo si $L_1 = L_2 = L$. La función es **continua** en a si además $f(a) = L$.

Fuentes de investigación

- [Khan Academy — Límites y continuidad](#)
- [Khan Academy — Tipos de discontinuidades](#)
- [MIT OCW — Single Variable Calculus \(notas en PDF\)](#)

4.2. Construcción de la función por tramos

A partir del RUT, `seleccionar_caso` elige uno de tres tipos según $d_8 \bmod 3$, y `construir_funcion` arma la descripción y el diccionario de tramos. El punto de análisis es $a = d_3$.

$d_8 \bmod 3$	Tipo	Función	Discontinuidad
$= 0$	Caso 1	$f(x) = \frac{(x-a)(x+d_1)}{x-a}$	Removable
$= 1$	Caso 2	$f(x) = \begin{cases} x+d_2 & x < a \\ x+d_4 & x \geq a \end{cases}$	Salto
$= 2$	Caso 3	$f(x) = \frac{d_5+1}{x-a}$	Infinita

4.3. Tipos de discontinuidad

4.3.1. Discontinuidad removable

El límite existe pero $f(a)$ no está definida. En el Caso 1:

$$\lim_{x \rightarrow a} \frac{(x-a)(x+d_1)}{x-a} = \lim_{x \rightarrow a} (x+d_1) = a+d_1$$

La simplificación cancela $(x-a)$ con $x \neq a$.

4.3.2. Discontinuidad de salto

Los límites laterales existen pero son distintos:

$$\lim_{x \rightarrow a^-} f(x) = a+d_2 \neq a+d_4 = \lim_{x \rightarrow a^+} f(x)$$

4.3.3. Discontinuidad infinita

La función crece sin acotamiento al acercarse a a (con numerador $d_5 + 1 > 0$):

$$\lim_{x \rightarrow a^-} \frac{d_5 + 1}{x - a} = -\infty, \quad \lim_{x \rightarrow a^+} \frac{d_5 + 1}{x - a} = +\infty$$

Existe una **asíntota vertical** en $x = a$.

```

1 def seleccionar_caso(d8):
2     """Determina el tipo de discontinuidad segun d8 mod 3."""
3     residuo = d8 % 3
4     if residuo == 0:
5         return "removable", f"d8={d8} -> mod 3 = 0 -> REMOVIBLE"
6     elif residuo == 1:
7         return "salto", f"d8={d8} -> mod 3 = 1 -> SALTO"
8     else:
9         return "infinita", f"d8={d8} -> mod 3 = 2 -> INFINITA"
10
11 def evaluar_funcion(x, tramos):
12     """Evalua f(x) segun los tramos. None si no esta definida."""
13     caso = tramos["tipo"]
14     a = tramos["a"]
15     if caso == "removable":
16         if x == a:
17             return None # no definida en x =
18                             a
19             return x + tramos["d1"] # (x-a)(x+d1)/(x-a)
20                             = x+d1
21     elif caso == "salto":
22         return x + tramos["d2"] if x < a else x + tramos["d4"]
23     elif caso == "infinita":
24         if x == a:
25             return None
26         return tramos["numerador"] / (x - a)
27     return None

```

Listing 8: Selección del caso y evaluación — `core/limites.py`

La función `calcular_limites` reúne todo el análisis: calcula límites laterales aproximando con $\varepsilon = 10^{-7}$, evalúa $f(a)$, decide si el límite existe, si la función es continua, clasifica el tipo de discontinuidad y arma la lista de pasos explicativos. Devuelve un diccionario que la capa de servicios traslada al objeto `LimitesAnalysis`.

4.4. Evidencia computacional: tabla de valores

La tabla de valores se genera en la capa de servicios mediante `_generar_tabla_valores`, que evalúa 11 puntos (cinco a cada lado de a más el propio a) con paso 0.2, contrastando así el límite analítico con evidencia numérica.

```

1 def _generar_tabla_valores(tramos, a, puntos=11):
2     """Genera tabla de valores cercanos al punto a (paso 0.2)."""
3     tabla = []
4     for i in range(puntos // 2, 0, -1): # lado izquierdo
5         x = a - i * 0.2
6         f_x = evaluar_funcion(x, tramos)
7         tabla.append({"x": round(x, 4),

```

```

8         "f_x": round(f_x, 4) if f_x is not None else
          None,
9         "lado": "izq"})
10     f_en_a = evaluar_funcion(a, tramos)           # punto a
11     tabla.append({"x": a,
12                  "f_x": round(f_en_a, 4) if f_en_a is not None
13                  else None,
14                  "lado": "a"})
15     for i in range(1, puntos // 2 + 1):         # lado derecho
16         x = a + i * 0.2
17         f_x = evaluar_funcion(x, tramos)
18         tabla.append({"x": round(x, 4),
19                      "f_x": round(f_x, 4) if f_x is not None else
20                      None,
21                      "lado": "der"})
22     return tabla

```

Listing 9: Generación de tabla de valores — `core/services.py`

4.4.1. Ejemplo de tabla (RUT 12.345.678-5, $a = 3$, caso infinita)

Para este RUT, $d_8 = 8 \Rightarrow 8 \bmod 3 = 2$ (infinita), $a = d_3 = 3$ y el numerador es $d_5 + 1 = 6$, es decir $f(x) = 6/(x - 3)$. La tabla generada por el código es:

x	$f(x)$	Lado
2.0	-6.0	← izq
2.2	-7.5	← izq
2.4	-10.0	← izq
2.6	-15.0	← izq
2.8	-30.0	← izq
3.0	No def.	a
3.2	30.0	der →
3.4	15.0	der →
3.6	10.0	der →
3.8	7.5	der →
4.0	6.0	der →

La tabla confirma que $f(x) \rightarrow -\infty$ por la izquierda y $f(x) \rightarrow +\infty$ por la derecha, evidenciando la **discontinuidad infinita** en $x = 3$.

5. Validadores, Constantes y Trazabilidad

5.1. Validadores reutilizables (`core/validators.py`)

Para no duplicar comprobaciones en cada módulo, `core/validators.py` centraliza la validación: formato del RUT (`validar_rut_str`), estructura de dígitos (`validar_digitos`), coeficientes de la cónica (`validar_coeficientes_conica`), punto de discontinuidad, tipo de caso, estructura de tramos y de la tabla de valores. Cada validador, ante un dato inválido, lanza la excepción correspondiente con un mensaje descriptivo.

5.2. Constantes (`core/constants.py`)

Todos los “números mágicos” (tolerancias como 10^{-9} y 10^{-10} , número máximo de iteraciones, decimales de visualización, parámetros del RUT, rangos de las gráficas, ϵ del límite y cantidad de puntos de la tabla) se definen en `core/constants.py`, lo que facilita ajustarlos sin tocar la lógica.

5.3. Sistema de logging (`core/log_service.py`)

El módulo `core/log_service.py` configura un logger con un *handler* en memoria (`InMemoryLogHandler`) que mantiene un búfer de eventos para mostrarlos en la interfaz, además de salida por consola y exportación a archivo (`export_logs`). Esto aporta trazabilidad: cada acción relevante de la aplicación queda registrada y puede revisarse desde el panel de logs.

6. Módulo de Interfaz Gráfica (`ui/`)

6.1. Tecnología utilizada: Tkinter

La interfaz se desarrolló con **Tkinter**, incluido en la instalación estándar de Python [12], por no requerir instalación adicional, permitir dibujar sobre un **Canvas** sin librerías matemáticas y ser suficientemente expresivo. La aplicación (`ui/app.py`) organiza el contenido en pestañas mediante un `ttk.Notebook`: panel de cónicas, panel de límites y panel de logs, sobre un tema oscuro definido en `ui/componentes.py`.

Fuentes de investigación

- [Documentación oficial de Tkinter \(Python 3 en español\)](#)
- [Recursos Python — Tutorial Tkinter en español](#)
- [Documentación Canvas de Tkinter](#)

6.2. Estructura de la interfaz

- `ui/app.py`: ventana principal, barra superior con el RUT validado compartido y `Notebook` con las pestañas.
- `ui/panel_conica.py`: ingreso de RUT, despliegue de la validación, ecuación general, clasificación, forma canónica con pasos y graficación. Invoca `analizar_conica`.
- `ui/panel_limites.py`: función por tramos, análisis de límites, tabla de valores y gráfica de la función. Invoca `analizar_limites`.
- `ui/panel_logs.py`: registro de eventos en tiempo real, con opción de exportar.
- `ui/componentes.py`: paleta de colores, fuentes y componentes reutilizables (cards, headers, barra de estado).

6.3. Graficación sin librerías matemáticas

La graficación se apoya en `core/graficas.py`. La función `puntos_grafica` genera los puntos de la cónica: circunferencias y elipses se trazan en forma paramétrica usando `cos_taylor` y `sin_taylor`; las hipérbolas con `cosh_taylor` y `sinh_taylor` (una lista de puntos por rama); y las parábolas evaluando el polinomio. Para los límites, `puntos_grafica_limite` construye los segmentos centrando la gráfica en $x = a$. Luego el panel convierte cada punto del plano matemático a píxeles del Canvas, invirtiendo el eje y .

```

1 def puntos_grafica(A, B, C, D, E, tipo, n=500):
2     """Genera puntos para dibujar la conica en un canvas."""
3     puntos = []
4     if tipo in ("Circunferencia", "Elipse"):
5         h, add_x = completar_cuadrado(A, C)
6         k, add_y = completar_cuadrado(B, D)
7         constante = -E - add_x - add_y
8         if constante <= 0:
9             return []
10        a = raiz_cuadrada_manual(constante / A)
11        b = raiz_cuadrada_manual(constante / B)
12        paso = 6.283185307179586 / n # 2*pi / n
13        for i in range(n + 1):
14            t = i * paso
15            puntos.append((h + a * cos_taylor(t), k + b *
16                          sin_taylor(t)))
17        # ... ramas de hiperbola con cosh/sinh, parabola con polinomio
18        ...
19    return puntos

```

Listing 10: Generación paramétrica de elipse/círculo — `core/graficas.py`

6.4. Campos de defensa oral

Los paneles incluyen campos inicialmente vacíos para que el estudiante los complete durante la defensa (Fase 4): para cónicas, centro, vértices, focos, radio/semiejes y directriz; para límites, límites por izquierda y derecha, existencia del límite, $f(a)$, continuidad, tipo de discontinuidad y justificación. La aplicación calcula internamente los valores correctos para poder contrastar las respuestas ingresadas.

7. Pruebas (test_services.py)

El archivo `test_services.py` ejercita la capa de servicios y el manejo de excepciones: prueba un RUT válido, RUTs con formato inválido (muy cortos) y con dígito verificador incorrecto, y verifica que se levanten `RUTInvalidoError`, `ConicaInvalidaError` o `LimiteInvalidoError` según corresponda. Esto valida que el flujo completo (validación, cónica y límites) se comporte de forma consistente y que los errores se propaguen con mensajes claros.

8. RUTs de Prueba Verificados

Los siguientes RUTs fueron ejecutados con el código y cubren las cuatro cónicas y los tres tipos de discontinuidad exigidos. La columna de observación indica el ajuste o regla que determina el resultado:

RUT	Cónica	Discontinuidad	Observación
12.345.678-5	Elipse	Infinita	RUT base ($a = d_3 = 3$, $f = 6/(x - 3)$)
11.345.678-7	Circunferencia	Infinita	$d_1 = d_2 \Rightarrow B = A$
13.456.789-9	Hipérbola	Removible	$d_8 = 9$ impar $\Rightarrow B = -B$
11.234.567-1	Parábola	Salto	$(d_5 + d_6) \bmod 3 = 0 \Rightarrow B = 0$
17.654.321-3	Hipérbola	Salto	$d_8 = 1$, residuo 1
14.725.836-4	Elipse	Removible	$d_8 = 6$, múltiplo de 3

9. Cumplimiento de Requerimientos

N°	Requerimiento	Estado
1	Validar RUT con algoritmo módulo 11	✓
2	Mostrar procedimiento de validación paso a paso	✓
3	Extraer dígitos del RUT	✓
4	Construir ecuación general paso a paso	✓
5	Clasificar la cónica automáticamente	✓
6	Mostrar forma canónica con procedimiento	✓
7	Mostrar procedimiento inverso (canónica $\rightarrow$ general)	✓
8	Graficar la cónica en el plano cartesiano	✓
9	Módulo de funciones por tramos	✓
10	Tabla de valores numéricos	✓
11	Campos vacíos para defensa oral	✓
12	Sin uso de numpy / math / sympy / scipy	✓
13	Código modular separado en archivos	✓
14	Capa de servicios y excepciones propias	✓
15	Sistema de logging y panel de trazabilidad	✓
16	Pruebas de la capa de servicios	✓
17	GitHub con commits por integrante	✓
18	Código de ética del grupo (etica.md)	✓

10. Conclusiones

El sistema desarrollado cumple con la totalidad de los requerimientos de la Evaluación Integrada de Desempeño N°1 del curso MAT1186. A través del proyecto se logró:

1. Aplicar el algoritmo oficial del módulo 11 para validar RUTs chilenos, mostrando el procedimiento matemático completo.
2. Implementar la construcción, clasificación y transformación de las cuatro secciones cónicas sin librerías matemáticas, incluyendo ajustes determinísticos sobre

los coeficientes para garantizar la cobertura de los cuatro tipos, y el procedimiento inverso canónica $\rightarrow$ general.

3. Implementar manualmente el método de Newton-Raphson para raíces cuadradas y las series de Taylor para funciones trigonométricas e hiperbólicas, reunidas en el módulo `core/modelos.py`.
4. Analizar funciones por tramos generadas desde el RUT, calculando límites laterales, clasificando discontinuidades y generando evidencia numérica mediante tablas de valores.
5. Estructurar el código en capas con una capa de servicios que orquesta el flujo, validadores reutilizables, excepciones propias, constantes centralizadas y un sistema de logging, lo que mejora la mantenibilidad y la trazabilidad.
6. Desarrollar una interfaz gráfica con Tkinter que integra todos los módulos mediante pestañas (cónicas, límites y logs) e incluye los campos de defensa oral.

Referencias

References

- [1] Servicio de Registro Civil e Identificación de Chile. *Dígito verificador del RUT — Algoritmo oficial*. Disponible en: <https://www.registrocivil.cl/principal/rut-y-digito-verificador>
- [2] Khan Academy. *Secciones cónicas*. Disponible en: <https://es.khanacademy.org/math/precalculus/x9e81a4f98389efdf:conics>
- [3] Khan Academy. *Completar el cuadrado — Revisión*. Disponible en: <https://es.khanacademy.org/math/algebra/x2f8bb11595b61c86:quadratics-multiplying-factoring/x2f8bb11595b61c86:complete-square/a/completing-the-square-review>
- [4] Khan Academy. *Límites y continuidad — Cálculo 1*. Disponible en: <https://es.khanacademy.org/math/calculus-1/cs1-limits-and-continuity>
- [5] Khan Academy. *Tipos de discontinuidades — AP Calculus AB*. Disponible en: <https://es.khanacademy.org/math/ap-calculus-ab/ab-limits-new/ab-1-10/a/types-of-discontinuities>
- [6] Khan Academy. *Series de Taylor en $x = 0$ (series de Maclaurin)*. Disponible en: <https://es.khanacademy.org/math/ap-calculus-bc/bc-series-new/bc-10-14/v/taylor-series-at-0-maclaurin>
- [7] OpenStax. *Álgebra y Trigonometría — Capítulos 10 y 11: Cónicas*. Disponible en: <https://openstax.org/books/algebra-y-trigonometria/pages/1-introduccion>
- [8] Wikipedia. *Método de Newton — Raíz cuadrada por el método de Herón*. Disponible en: https://es.wikipedia.org/wiki/M%C3%A9todo_de_Newton

- [9] Wikipedia. *Serie de Taylor*. Disponible en: https://es.wikipedia.org/wiki/Serie_de_Taylor
- [10] Wikipedia. *Sección cónica*. Disponible en: https://es.wikipedia.org/wiki/Secci%C3%B3n_c%C3%B3nica
- [11] Wikipedia. *Número de identificación tributaria (Chile)*. Disponible en: [https://es.wikipedia.org/wiki/N%C3%BAmero_de_identificaci%C3%B3n_tributaria_\(Chile\)](https://es.wikipedia.org/wiki/N%C3%BAmero_de_identificaci%C3%B3n_tributaria_(Chile))
- [12] Python Software Foundation. *tkinter — Interfaz de Python para Tk*. Disponible en: <https://docs.python.org/es/3/library/tkinter.html>
- [13] GitHub, Inc. *Documentación de GitHub — Primeros pasos*. Disponible en: <https://docs.github.com/es/get-started>
- [14] MIT OpenCourseWare. *18.01 — Single Variable Calculus (Fall 2006)*. Disponible en: <https://ocw.mit.edu/courses/18-01-single-variable-calculus-fall-2006/>